CLASE 25

SQLite con Python

Conectando codigo con datos persistentes

Hoy aprenderan:

- Modulo sqlite3 (viene con Python)
- Conectar Python con bases de datos
- Ejecutar SQL desde codigo Python
- Parametros seguros (SQL injection)
- Proyecto: Sistema con datos persistentes

Finalmente: datos que NO se borran al cerrar

El panorama completo

Lo que ya saben:

PYTHON

- Clases y objetos
- Logica y funciones
- Interfaces de usuario
- Datos en memoria (temporales)

SQL

- CREATE TABLE, INSERT
- SELECT, WHERE, JOIN
- UPDATE, DELETE
- Datos persistentes (en disco)

sqlite3 - El puente entre Python y SQL

Viene incluido con Python - no hay que instalar nada. Hoy aprenden a usar ese puente.

Paso 1: Conectar a la base de datos

1. Importar sqlite3
2. Conectar a la base de datos (.db)
3. Crear un cursor (ejecuta comandos)
4. Ejecutar comandos SQL
5. Cerrar la conexion

Analogia del restaurante

Conexion	=	Entrar al restaurante
Cursor	=	El mesero
execute()	=	Hacer un pedido
fetchall()	=	Recibir la comida
close()	=	Pagar e irte

Si el archivo .db no existe, SQLite lo CREA automaticamente

Vamos a VS Code a ver como se hace paso a paso

El cursor - Tu asistente de SQL

El cursor es el que realmente ejecuta los comandos

`execute(sql)`	Ejecuta UN comando SQL
`executemany(sql, datos)`	Ejecuta MUCHOS de una vez
`fetchone()`	Trae UNA fila
`fetchall()`	Trae TODAS las filas
`fetchmany(n)`	Trae N filas

Cuando usar cada fetch:

`fetchone()`	Cuando esperas 1 resultado (buscar por ID)
`fetchall()`	Cuando quieres todo (listar todos)
`fetchmany(10)`	Cuando hay muchos y quieres de a poco

Forma segura: Context Manager (with)

PROBLEMA

Si hay un error antes de close(), la conexion queda abierta indefinidamente.

Es como irse del restaurante sin pagar.

SOLUCION: usar with

"with" cierra la conexion automaticamente. Incluso si hay errores.

Igual que con archivos:
with open('archivo.txt') as f:

Beneficios del with:

- ✓ Cierra la conexion automaticamente al salir del bloque
- ✓ Funciona incluso si hay errores (excepciones)
- ✓ Codigo mas limpio y profesional

SIEMPRE usen "with" para conexiones de base de datos. Es la forma profesional.

SELECT desde Python

El patron basico para consultas:

1. Conectar con with sqlite3.connect()
2. Crear cursor
3. cursor.execute('SELECT ...')
4. cursor.fetchall() o fetchone()
5. Iterar los resultados con for

Los resultados vienen como TUPLAS:

Si haces SELECT id, titulo, autor ...
Cada fila es: (1, '1984', 'George Orwell')
Accedes por indice: fila[0] = id, fila[1] = titulo, fila[2] = autor

El orden de los elementos = orden de las columnas en tu SELECT

Resultados como diccionarios

Por defecto: TUPLAS

fila = (1, '1984', 'Orwell')
titulo = fila[1]

Hay que recordar el indice de cada columna

Mejor: DICCIONARIOS

fila = {'id': 1, 'titulo': '1984', ...}
titulo = fila['titulo']

Acceso por nombre, mucho mas claro

Como obtener diccionarios:

Agregar UNA linea despues de conectar:

```
conexion.row_factory = sqlite3.Row
```

sqlite3.Row permite acceso por nombre Y por indice. Mi recomendacion: usarlo siempre.

EJERCICIO RAPIDO: Consultas

Crear un archivo que haga estas consultas:

1. Mostrar todos los usuarios (nombre y email)
2. Contar cuantos libros hay en total
   Pista: SELECT COUNT(*) ...
3. Mostrar los libros del siglo XX (anio >= 1900 y < 2000)
4. Buscar libros que contengan "el" en el titulo
   Pista: WHERE titulo LIKE '%el%'

Tiempo: 8 minutos | Pueden usar row_factory para resultados mas legibles

Modificar datos: INSERT, UPDATE, DELETE

Para cambios en la base de datos, necesitas commit()

INSERT

Agregar registros nuevos

UPDATE

Modificar registros existentes

DELETE

Eliminar registros

Por que commit()?

- Los cambios son "temporales" hasta que confirmas
- Si algo falla, puedes hacer rollback() (deshacer todo)
- commit() hace los cambios PERMANENTES en disco

Sin commit(): los cambios se PIERDEN al cerrar la conexion

SELECT no necesita commit (solo lee). INSERT, UPDATE, DELETE SI necesitan commit (modifican).

Obtener el ID del registro insertado

Despues de INSERT, a veces necesitas saber el ID asignado

`cursor.lastrowid`

El ID del ultimo registro insertado.

Util para:
- Confirmar que se creo el registro
- Relacionar con otras tablas
- Mostrar "Libro #X creado"

`cursor.rowcount`

Cuantas filas afecto la ultima operacion.

Util para:
- Saber si UPDATE cambio algo
- Saber si DELETE borro algo
- Validar que la operacion funciono

Patron tipico:

Despues de INSERT: nuevo_id = cursor.lastrowid
Despues de UPDATE/DELETE: if cursor.rowcount > 0: # Se modifico algo

lastrowid solo funciona con tablas que tienen PRIMARY KEY autoincremental

EJERCICIO: Operaciones CRUD

Practicar INSERT, UPDATE, DELETE con commit:

1. Insertar 2 libros nuevos y mostrar sus IDs
2. Actualizar el titulo de un libro existente
3. Marcar un libro como no disponible (disponible = 0)
4. Eliminar un libro y verificar con rowcount
5. Listar todos los libros para verificar los cambios

Recuerden: conexion.commit() despues de cada cambio. Sin commit = cambios perdidos.

Tiempo: 5 minutos

BREAK

10 MINUTOS

Volvemos con:
Parametros seguros (muy importante)
Proyecto final con base de datos

PELIGRO: SQL Injection

Que pasa si el usuario escribe algo malicioso?

Codigo peligroso (concatenacion de strings):

Si usas f-strings o + para meter datos del usuario en SQL,
un atacante puede escribir: ' OR '1'='1

Y el SQL retorna TODOS los registros de la tabla!

Peor aun: podrian escribir

'; DROP TABLE usuarios; --

Esto BORRA toda la tabla de usuarios. Datos irrecuperables.

NUNCA concatenes strings del usuario en SQL
Siempre usa parametros (siguiente slide)

Solucion: Parametros (placeholders)

PELIGROSO (concatenacion)

```
f"SELECT * FROM usuarios
  WHERE id = {id}"
```

El valor se pega directo en el SQL

SEGURO (parametros ?)

```
"SELECT * FROM usuarios
  WHERE id = ?", (id,)
```

SQLite escapa caracteres peligrosos

Como funciona:

- ✓ El ? es un marcador de posicion (placeholder)
- ✓ El valor real va en una tupla separada
- ✓ SQLite se encarga de limpiar/escapar caracteres peligrosos
- ✓ Imposible inyectar codigo SQL malicioso

SIEMPRE usa ? para valores que vienen del usuario. Nota: (id,) lleva coma - es una tupla de 1 elemento.

Parametros nombrados

Con ? (posicional)

VALUES (?, ?, ?)
+ tupla: (titulo, autor, anio)

Orden importa: 1er ? = titulo, etc.

Con :nombre (nombrado)

VALUES (:titulo, :autor, :anio)
+ diccionario: {'titulo': ..., 'autor': ...}

Orden NO importa, usa nombres

Ventajas de parametros nombrados:

- Mas claro cuando hay muchos parametros
- No importa el orden de los valores
- Puedes reusar el mismo valor en varios lugares
- Se integra bien con diccionarios de Python

Ambos metodos (? y :nombre) son igualmente seguros. Usa el que prefieras.

Patron completo: CRUD con Python

C CREATE (Insertar)

```
execute('INSERT ...', params)
commit()
nuevo_id = lastrowid
```

R READ (Consultar)

```
execute('SELECT ...', params)
resultado = fetchone()
resultados = fetchall()
```

U UPDATE (Actualizar)

```
execute('UPDATE ...', params)
commit()
exito = rowcount > 0
```

D DELETE (Eliminar)

```
execute('DELETE ...', params)
commit()
exito = rowcount > 0
```

Patron: execute() + parametros seguros + commit() (si modifica datos)

Veamos el patron completo en VS Code con funciones

PROYECTO: Biblioteca con base de datos

Crear una clase BibliotecaDB que encapsule toda la logica de BD:

1. agregar_libro(titulo, autor, isbn, anio) → retorna ID
2. obtener_libro(id) → retorna diccionario o None
3. listar_libros() → retorna lista de diccionarios
4. buscar_libros(termino) → busca en titulo y autor
5. actualizar_disponibilidad(id, disponible) → True/False
6. eliminar_libro(id) → True/False

Requisitos:

with → para conexiones

row_factory → para resultados legibles

Parametros ? → para todos los valores

commit() → en operaciones de escritura

Tiempo: 20 minutos | Pueden trabajar en parejas

Resumen - SQLite con Python

Concepto	Como se hace
Conectar	`sqlite3.connect('archivo.db')`
Cursor	`conexion.cursor()`
Ejecutar	`cursor.execute(sql, params)`
Obtener resultados	`fetchone() / fetchall()`
Guardar cambios	`conexion.commit()`
Parametros seguros	`? o :nombre`
Diccionarios	`row_factory = sqlite3.Row`

Reglas de oro:

SIEMPRE usar "with" para conexiones

SIEMPRE usar parametros (?) para datos de usuario

SIEMPRE commit() despues de INSERT / UPDATE / DELETE

Proxima clase: Virtual Environments + pip - "Gestionar dependencias como profesionales"